MedRAG

Project Documentation

A Medical Agentic Multimodal Retrieval-Augmented Generation System for Evidence-Grounded Clinical Decision Support

A complete explanation of every file in the project: what it is, why it was created and what it does.

Danish Ali Raza · Registration 2021-GCUF-063725
MS Computer Science, Govt. College University Faisalabad
10 November 2026

Contents

1. Introduction 3

2. How the system works (the process) 4

Example: de-identification of a synthetic chest X-ray . . . 4

Agent state machine . . . 5

Report lifecycle . . . 5

The clinical workspace . . . 5

3. Technology choices and reasons 6

Design principles . . . 6

4. Repository structure 7

5. File-by-file reference 8

5.1 Project configuration and tooling . . . 8

5.2 Documentation (docs/) . . . 10

5.3 Data (data/) . . . 11

5.4 Core package (medrag/) . . . 12

5.5 Retrieval (medrag/retrieval/) . . . 15

5.6 Imaging (medrag/imaging/) . . . 18

5.7 De-identification (medrag/deid/) . . . 20

5.8 Agent (medrag/agent/) . . . 22

5.9 Reports (medrag/reports/) . . . 24

5.10 Storage (medrag/storage/) . . . 25

5.11 Web API (medrag/api/) . . . 27

5.12 Evaluation (medrag/eval/) . . . 28

5.13 Web workspace (medrag/web/) . . . 29

5.14 Tests (tests/) . . . 30

6. Database tables 33

Files written at run time (inside var/) . . . 33

7. API endpoints 33

8. Configuration 34

9. How to run 34

10. How the work was verified 35

Offline retrieval results (harness check) . . . 35

Offline clinical-drafting results (harness check) . . . 35

11. Limitations and next steps 35

1. Introduction

MedRAG is the working implementation described in the MS synopsis. Given a clinical image (chest radiograph, retinal fundus photograph or histopathology slide) and a clinical question, it drafts a structured diagnostic report. Every statement in the report cites trusted evidence, and the system checks its own draft against that evidence before showing it. All patient identifiers are removed before anything is stored, and only a credentialed human clinician can finalise a report.

This document is a complete guide to the source code. Section 2 explains the process the system follows. Section 3 explains the technology choices and the reasons for them. Sections 4 and 5 go through **every file** in the repository: what it is, why it was created and what it does, followed by a list of its classes and functions taken directly from the code. The later sections cover the database, the API, configuration, how to run the project, how it was verified, and its limitations.

Measure	Value
Files in the repository	93
Python source (application)	40 files, 3,272 lines
Automated tests	79 test functions, 117 test cases once parameters are expanded, 808 lines
Web workspace (HTML / CSS / JS)	489 lines
Knowledge-base documents	15 (+ any uploaded by an admin)
Imaging modalities	3 (chest X-ray, fundus, histopathology)

Important: the system is research software and decision support only. It is not a medical device. The knowledge base contains educational summaries written for development, and all images are synthetic.

2. How the system works (the process)

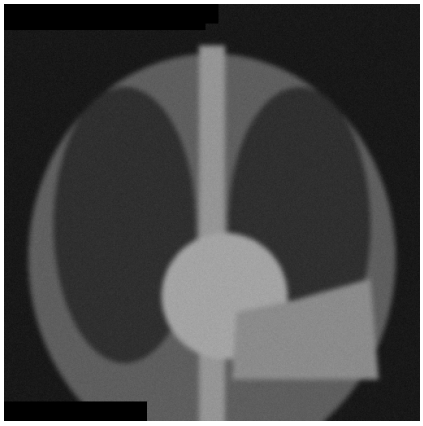
A single case passes through seven steps. Steps 4 to 6 run inside the agent and can repeat.

#	Step	What happens	Main files
1	Register patient	The hospital ID becomes an irreversible HMAC pseudonym and the age becomes an age band. The raw ID is never stored.	deid/hashing.py, service.py
2	Upload and de-identify	Image metadata is dropped, burned-in text is blacked out and identifiers in the clinical note are redacted. Only the clean image is stored.	deid/image.py, deid/text.py, storage/
3	Ask	The clinician enters a question and picks a variant (normally the full agent). Identifiers in the question are scrubbed.	api/app.py, web/app.js
4	Retrieve	Hybrid search: metadata filter, dense and sparse search, reciprocal rank fusion, re-ranking, top 5 passages.	retrieval/
5	Read image	The vision model lists the visible findings. Similar signed-off cases are retrieved. Retrieval runs again with the finding names added.	imaging/, agent/graph.py
6	Draft and verify	A sectioned report is drafted with citations, then every claim is checked against its cited passage. Unsupported claims trigger a new search (at most 3 rounds), otherwise the draft is flagged for review.	agent/generator.py, agent/verifier.py
7	Sign off	Only a human clinician may sign off or reject (with a reason). The PDF loses its DRAFT watermark only after sign-off. Every action is in the hash-chained audit log.	reports/, storage/db.py

Example: de-identification of a synthetic chest X-ray



Uploaded image: a fake name and MRN are burned into the pixels



Stored image: text regions masked, metadata removed

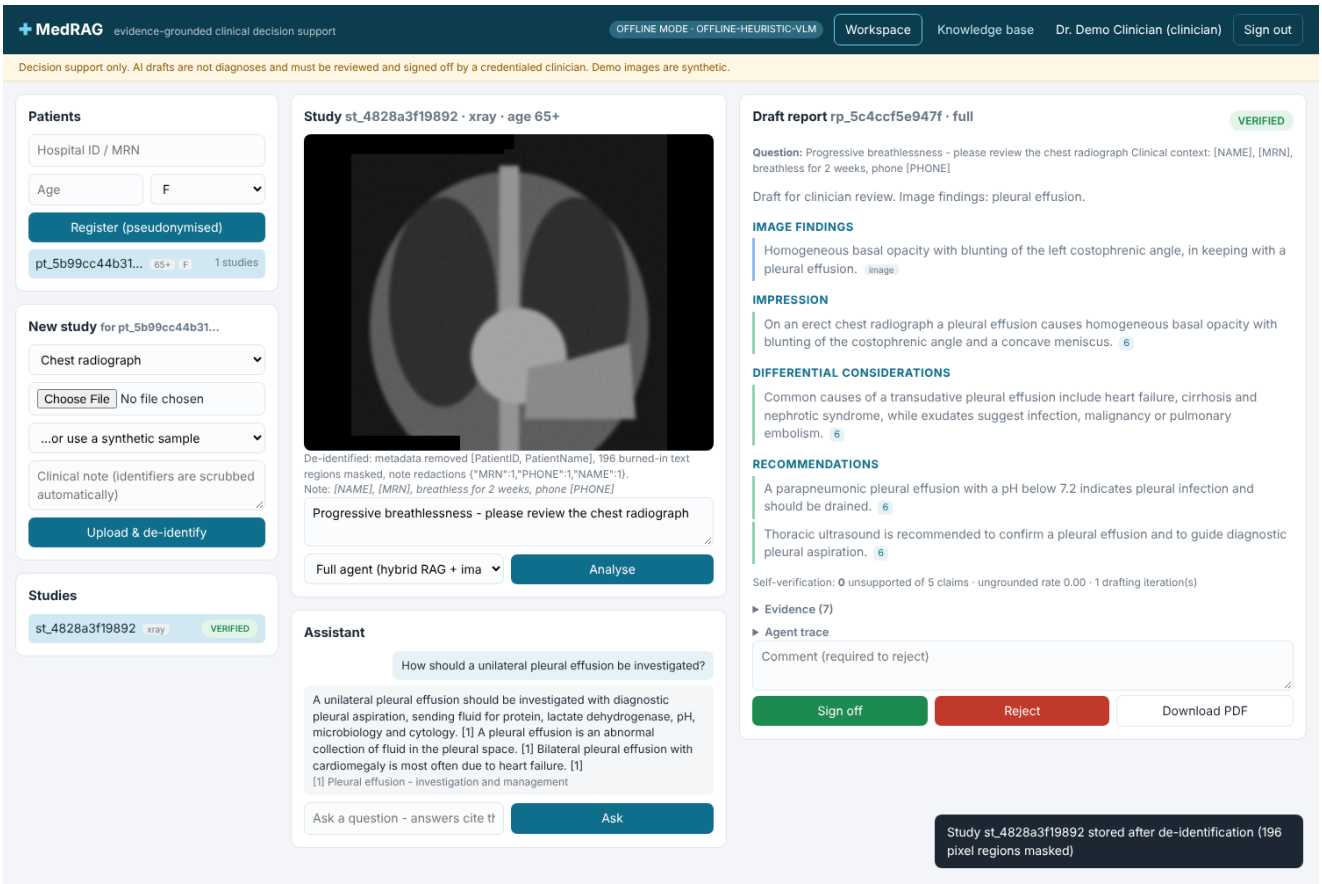
Agent state machine

```
route ──► retrieve ──► inspect_image ──► retrieve (+ image concepts) ──► draft ──► verify
            ▲                                                                       │
            └──────── reformulated query (unsupported concepts) ◄── fails ──────────┘
                                  after max iterations ──► needs_review (flagged)
Outcomes: verified | needs_review | unverified (ablation)   —   never signed_off
```

Report lifecycle

```
draft ──(agent)──► verified ──┐
      └─(agent)──► needs_review └──(human clinician only)──► signed_off   (terminal)
                                                          └─► rejected    (terminal, reason required)
```

The clinical workspace



Left: patients and studies. Centre: the de-identified image, the question and the assistant chat. Right: the cited draft with its verification result, the sign-off and reject buttons and the PDF download.

3. Technology choices and reasons

Technology	Used for	Why it was chosen
Python 3.10+	All back-end code	The standard language for machine learning and medical-imaging libraries.
FastAPI + Uvicorn	Web API	Automatic input validation and interactive API docs (/docs), fast, and well suited to ML services.
Pydantic	Request models	Rejects bad input (ages, variants, lengths) before it reaches the logic.
SQLite	Records and audit log	No server needed, a single file, supports triggers for the append-only log. Can be swapped for PostgreSQL.
NumPy	Vector search, image analysis	Fast array maths with no GPU required.
Pillow	Image decoding, re-encoding, synthetic images	Lightweight; re-encoding through it removes metadata.
ReportLab	PDF reports	Pure Python, precise layout control, watermarks.
BM25 (own code)	Sparse search	Exact-term matching for drug names and abbreviations; small enough to audit.
BAAI/bge-m3	Dense embeddings (real mode)	A strong retrieval encoder named in the synopsis.
bge-reranker-v2-m3	Cross-encoder re-ranking (real mode)	Improves precision of the top passages.
Qwen2.5-VL via vLLM	Vision-language model (real mode)	Open-weights VLM that is self-hostable; vLLM gives an OpenAI-compatible API on local hardware, as the data-use agreements require.
BiomedCLIP	Image embeddings (real mode)	A biomedical image encoder for similar-case search.
Qdrant	Vector database (real mode)	Production-grade, self-hosted vector search.
Tesseract OCR	Burned-in text (real mode)	Adds OCR word boxes to the stroke detector.
Vanilla HTML/CSS/JS	Clinical workspace	No build step and no framework to install, so it works offline on hospital servers.
pytest, ruff, GitHub Actions	Quality	Automated proof that the safeguards hold, and a consistent code style.
Docker / Compose	Deployment	Reproducible, self-hosted stack with an optional GPU profile.

Design principles

- **Safety in structure, not in good intentions.** De-identification happens inside the service at ingestion; the sign-off rule lives below the API; the vector store refuses non-allow-listed fields.
- **Two modes, one pipeline.** Every model has an offline stand-in with the same interface, so the complete control flow can be developed and tested without a GPU. Heavy libraries load only when first used (lazy imports).
- **Everything inspectable.** The agent is a plain state machine that writes a trace; every action goes into a tamper-evident audit log.
- **Honest evaluation.** Metrics and release gates are fixed before the run, and offline figures are labelled as harness checks, not clinical results.

4. Repository structure

```
MEDRAG/
├── pyproject.toml, Makefile, Dockerfile, docker-compose.yml, .env.example, .gitignore, README.md
├── .github/workflows/ci.yml      continuous integration
├── docs/                         ARCHITECTURE, EVALUATION, PLAN, screenshot
├── data/
│   ├── knowledge_base/           15 guideline summaries (markdown)
│   ├── gold/                     retrieval_gold.json, report_gold.json
│   └── samples/                  10 synthetic images with fake burned-in IDs
├── medrag/                       the application package
│   ├── config.py, textutils.py, llm.py, service.py, cli.py
│   ├── retrieval/                chunking, embeddings, sparse, vector_store, fusion, rerank, pipeline
│   ├── imaging/                  synthetic, analysis, vlm, encoder
│   ├── deid/                     hashing, text, image
│   ├── agent/                    state, generator, verifier, graph
│   ├── reports/                  lifecycle, document
│   ├── storage/                  db, objects
│   ├── api/                      auth, app
│   ├── eval/                     metrics, runner
│   └── web/                      index.html, style.css, app.js
└── tests/                        conftest + 9 test modules
var/ (created at run time, not committed): medrag.sqlite3, objects/, index/, kb_uploads/
```

Dependency direction. The web workspace calls the API; the API calls the service; the service uses de-identification, storage, retrieval, the agent and the reports module; the agent uses retrieval, imaging and its generator and verifier. Lower layers never import higher ones, so each can be tested on its own.

5. File-by-file reference

Each entry gives **what** the file is, **why** it was created, and **what it does**. For Python files, a table of classes, functions and constants follows, extracted automatically from the code (so it always matches). Line counts are given for orientation.

5.1 Project configuration and tooling

Files at the repository root that describe, install, build, run and check the project.

`pyproject.toml` · 55 lines

What it is. The Python package definition: name, version, dependencies, optional dependency groups, the medrag command and tool settings for pytest and ruff.

Why it was created. A single standard file (PEP 621) means the project installs with one command (pip install -e .). The heavy machine-learning libraries are kept in an optional real group, so a laptop without a GPU only installs the light core.

What it does

- Core dependencies: FastAPI, Uvicorn, Pydantic, python-multipart (file uploads), NumPy, Pillow (images), ReportLab (PDF).
- Optional groups: dev (pytest, httpx, ruff), graph (LangGraph), real (sentence-transformers, open_clip, torch, qdrant-client, pytesseract, pydicom).
- Registers the console command medrag = medrag.cli:main.
- Ships the web files (medrag/web/*) inside the package so the UI works after installation.
- Configures pytest (tests folder, quiet output) and ruff (line length 120, error, import-order, bug and upgrade rules).

`.gitignore` · 11 lines

What it is. Tells Git which generated files never to commit.

Why it was created. Runtime data (the var/ folder holds the database and uploaded images) must never be committed. It is patient-derived data, even after de-identification. The virtual environment, caches, evaluation output and the private .env file are also excluded.

What it does

- Ignores .venv/, __pycache__/, test and lint caches, /var/, /eval_results/, build artefacts and .env.

`.env.example` · 15 lines

What it is. A template for the environment variables that configure a deployment.

Why it was created. Secrets must not live in code or in Git. Users copy this file to .env and change the secret key and patient salt. The file also documents every real-mode setting in one place.

What it does

- Sets the mode (offline / real), secret key, patient-hash salt and demo-user seeding.
- Lists the commented-out real-mode settings: LLM URL and model, embedding, re-ranker and image-encoder models, Qdrant, and the similarity model used in evaluation.

`Makefile` · 35 lines

What it is. Short commands for the common tasks.

Why it was created. A supervisor or examiner can run the project without remembering long commands.

What it does

- make install: create .venv and install with dev tools.
- make run: start the web server on port 8000.
- make test, make lint: run the test suite and the linter.
- make eval: run the evaluation with release gates (fails if a gate fails).
- make demo: one complete case in the terminal. make samples: write the synthetic images. make docker: start with Docker Compose. make clean: delete runtime data.

`Dockerfile` · 21 lines

What it is. Instructions to build a container image of the application.

Why it was created. Containers make the deployment reproducible and self-hostable, as the synopsis requires. The same image works on a hospital server with no internet access.

What it does

- Starts from python:3.12-slim and installs Tesseract OCR (used in real mode to find burned-in text).
- Copies the package and data, then installs it. The build argument EXTRAS=real adds the GPU-mode libraries.
- Runs as a non-root user, keeps data in the /data volume, exposes port 8000 and adds a health check on /api/health.

`docker-compose.yml` · 36 lines

What it is. Defines the services that run together.

Why it was created. One command starts the whole stack. A gpu profile adds the model server and vector database only when real mode is wanted.

What it does

- medrag: the application, with a persistent data volume and an optional .env file.
- qdrant (gpu profile): the production vector database.
- vllm (gpu profile): serves Qwen2.5-VL-7B-Instruct behind an OpenAI-compatible API on port 8001, reserving one NVIDIA GPU. Model weights are cached in a volume.

`.github/workflows/ci.yml` · 26 lines

What it is. The continuous-integration pipeline on GitHub Actions.

Why it was created. Every change is checked automatically, so a regression in safety or quality is caught before it is merged. The synopsis treats a rise in the hallucination rate as release-blocking, and this pipeline enforces that.

What it does

- Runs on every push and pull request with Python 3.10 and 3.12.
- Installs the package, runs ruff and the full pytest suite.
- Runs medrag eval --strict, which fails the build if Recall@5 < 0.80 or the full system's ungrounded-claim rate > 0.10.

- Uploads the evaluation results as a build artefact.

`README.md` · 116 lines

What it is. The front page of the repository.

Why it was created. It is the first thing a reader sees. It explains what the system is, how each synopsis component maps to code, how to run it, and its limitations.

What it does

- Component-to-code table, quick start, demo walk-through, real-mode instructions, architecture diagram and important disclaimers.

5.2 Documentation (docs/)

Longer explanations kept next to the code so that they stay up to date.

`docs/ARCHITECTURE.md` · 71 lines

What it is. Describes the layers, every API endpoint, the agent control flow and the privacy guarantees.

Why it was created. Examiners and future contributors need a precise map of the system. Each privacy guarantee is linked to the test that proves it.

What it does

- Layer table, endpoint and permission table, agent flow diagram, and a guarantee-to-test table.

`docs/EVALUATION.md` · 50 lines

What it is. Defines every metric, the four ablation variants, the release gates and how to read offline numbers.

Why it was created. Metrics must be defined before results are reported, so that the thesis evaluation is reproducible and honest.

What it does

- Metric formulas, ablation matrix, thresholds, and the steps needed for real thesis experiments.

`docs/PLAN.md` · 33 lines

What it is. Traces each research question to its objective, implementation and evidence, and gives the phase status.

Why it was created. It connects the synopsis to the code and shows which work is done and which remains.

What it does

- RQ1 to RQ4 traceability table, phase status table and known limitations.

`docs/workspace.png`

What it is. A screenshot of the clinical workspace, taken by an automated browser test.

Why it was created. Shows what the running system looks like without having to install it.

5.3 Data (data/)

Inputs the system reads: the knowledge base, the evaluation gold sets and sample images.

data/knowledge_base/*.md (15 files)

What it is. The sample evidence corpus that the system retrieves from and cites: community-acquired pneumonia, pleural effusion, pneumothorax, heart failure and cardiomegaly, tuberculosis, lung nodule, normal chest X-ray, diabetic retinopathy, glaucoma, age-related macular degeneration, normal fundus, invasive breast carcinoma, colorectal adenocarcinoma, benign tissue, and safe use of AI.

Why it was created. A RAG system can only cite what it has been given. These short educational summaries, written for development, cover every finding the image module can report, so every image finding has evidence to cite. They must be replaced with licensed NICE / WHO / PubMed Central text for the thesis experiments.

What it does

- Each file starts with front matter: id, title, source, modality (xray, fundus, histopathology or general) and optionally age_band.
- The modality tag drives the metadata filter, so a fundus study is never answered with chest X-ray guidance.
- Sentences are written so that the drafting step can quote them: definition, imaging signs, investigation, differential diagnosis and recommendations.

data/gold/retrieval_gold.json · 28 lines

What it is. 26 test queries, each paired with the knowledge-base documents that should be retrieved.

Why it was created. Recall@K and MRR need a ground truth of correct answers. It is kept separate from the knowledge base so that the system is not tested on its own index.

What it does

- Each item: query and the list of relevant document ids. Some queries have two relevant documents.

data/gold/report_gold.json · 44 lines

What it is. 14 synthetic imaging cases across the three modalities, each with a clinical question, an expected key finding and a clinician-style reference report.

Why it was created. Used to measure the ungrounded-claim rate, report similarity and finding accuracy for each system variant. A test checks that no reference text appears in the knowledge base.

What it does

- Fields: modality, finding (which synthetic image to generate), seed, expected_concept, question, reference.

data/samples/*.png (10 files)

What it is. Ready-made synthetic images: five chest X-rays (normal, consolidation, effusion, cardiomegaly, pneumothorax), three fundus photos (normal, diabetic retinopathy, glaucoma) and two histopathology slides (benign, carcinoma).

Why it was created. They let anyone try the upload screen with a file. Each has a FAKE name and MRN burned into the pixels and stored in the metadata, to show that de-identification removes them. They are cartoons, not patient images.

What it does

- Generated by medrag samples from medrag/imaging/synthetic.py.

5.4 Core package (medrag/)

Shared building blocks used by every other module.

`medrag/__init__.py` · 7 lines

What it is. Marks the folder as a Python package and holds the version number.

Why it was created. The version is shown by the health endpoint and the API documentation.

What it does

- Defines __version__ = "1.0.0" and a docstring stating that the system is decision support only.

`medrag/config.py` · 85 lines

What it is. All runtime settings, read from MEDRAG_* environment variables, in one Settings dataclass.

Why it was created. One place for configuration makes the offline/real switch a single variable (MEDRAG_MODE), and lets tests create isolated settings, such as a temporary data folder.

What it does

- Paths: data folder, knowledge-base folder, and derived database, object and index paths.
- Security: secret key for tokens, salt for patient hashes, token lifetime.
- Retrieval: chunk size (500), overlap (60), top-K (5), candidate pool (20), RRF constant (60), vector backend and Qdrant URL.
- Agent: maximum iterations (3) and support threshold (0.5).
- Model names for real mode, LLM endpoint and timeout, and the demo-user seeding switch.
- get_settings() caches one Settings instance for the whole process.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const ROOT_DIR
def _env(name: str, default: str)
class Settings
  .offline(self)
  .db_path(self)
  .object_dir(self)
  .index_dir(self)
def get_settings()
```

`medrag/textutils.py` · 55 lines

What it is. Small text helpers: tokenizer, stopword list, light stemmer, sentence splitter and the overlap ratio.

Why it was created. Retrieval, re-ranking, drafting and verification must agree on what a "term" is. Sharing one implementation keeps them consistent, and it has no dependencies.

What it does

- tokenize(): lower-cases text and extracts words, keeping hyphenated terms such as "x-ray".
- content_terms(): removes stopwords and stems words, so that "opacities" and "opacity" match.
- split_sentences(): splits text into sentences for chunking and drafting.
- overlap_ratio(claim, evidence): the fraction of a claim's key terms found in the evidence. It is the core of the verifier.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const _TOKEN_RE
const _SENT_RE
const STOPWORDS
const _SUFFIXES
def stem(token: str)
def tokenize(text: str)
def content_terms(text: str) : Lower-cased, stemmed tokens with stopwords removed.
def split_sentences(text: str)
def overlap_ratio(claim: str, evidence: str) : Fraction of the claim's content terms that occur in the evidence.
```

medrag/llm.py · 53 lines

What it is. A minimal client for any OpenAI-compatible chat endpoint (vLLM, llama.cpp, TGI).

Why it was created. Real mode talks to a model served on the institution's own hardware. Using only the standard library avoids an extra dependency and makes the network call easy to audit.

What it does

- ChatClient.chat(): sends messages and returns the reply text, raising LLMError if the server is unreachable.
- image_part(): wraps PNG bytes as a base64 data URL for vision models.
- extract_json(): pulls the first JSON object out of a model reply, including from ``` code blocks.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class LLMError
class ChatClient
  .__init__(self, settings: Settings | None=None)
  .chat(self, messages: list[dict], max_tokens: int=1024, temperature: flo…)
def image_part(png_bytes: bytes)
def extract_json(text: str) : Pull the first JSON object/array out of a model response.
```

medrag/service.py · 225 lines

What it is. The application service: the one place where storage, retrieval, the agent, de-identification and the report lifecycle meet.

Why it was created. Putting all business rules here, below the web layer, means that no caller (web, CLI or test) can skip de-identification, audit logging or the sign-off rules.

What it does

- Start-up: creates folders and the database, then loads the search index if the knowledge-base fingerprint is unchanged, otherwise rebuilds it. Builds the similar-case index from stored studies and seeds demo users on first run.
- register_patient(): turns the hospital ID into a pseudonym, age into an age band, normalises sex, and audits the action.
- ingest_study(): validates modality and patient, de-identifies the image, scrubs the clinical note, stores the clean image in the object store, records the study, adds it to the similar-case index, and audits it.
- analyze(): scrubs the question, runs the agent for the chosen variant, stores the result as a draft, then moves it to verified or needs_review as the agent actor (audited).
- sign_off() / reject(): check the lifecycle rules for the human actor. Refusals are audited and raised. Success updates the report, and a signed-off study enters the similar-case index with its findings.
- agent_attempt_signoff(): exists only to demonstrate and test that the agent is refused.

- report_pdf(): renders the PDF with the study image, and audits the export.
- add_kb_document(): scrubs, chunks and indexes a new guideline, saves it to kb_uploads/ so it survives index rebuilds, and updates the fingerprint.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const MODALITIES
const AGENT
class NotFound
class MedRAGService
  .__init__(self, settings: Settings | None=None, db: Database | None=None)
  ._kb_fingerprint(self)
  ._load_or_build_index(self) : Reuse the persisted index only if the knowledge-base sources are unchanged.
  .seed_users(self)
  ._rebuild_case_index(self)
  .register_patient(self, user: dict, patient_id: str, age: int | None, sex: str)
  .ingest_study(self, user: dict, patient_ref: str, modality: str, image_bytes: by…)
  .study_image(self, sid: str)
  .analyze(self, user: dict, sid: str, question: str, variant: str='full')
  .ask(self, user: dict, question: str, modality: str | None=None)
  ._actor(self, user: dict)
  .sign_off(self, user: dict, rid: str, comment: str='')
  .reject(self, user: dict, rid: str, reason: str)
  ._decide(self, user: dict, rid: str, target: ReportStatus, comment: str)
  .agent_attempt_signoff(self, rid: str) : Exists only to demonstrate (and test) that the agent cannot sign off.
  .report_pdf(self, user: dict, rid: str)
  .add_kb_document(self, user: dict, title: str, text: str, source: str, modality: li…)
```

medrag/cli.py · 112 lines

What it is. The medrag command-line tool.

Why it was created. Gives one entry point for serving, indexing, evaluating and demonstrating, which CI, Docker and the Makefile all use.

What it does

- serve: starts Uvicorn with the FastAPI app.
- ingest: rebuilds the knowledge-base index and prints statistics.
- eval: runs the evaluation, prints the Markdown report, and with --strict exits with an error if a gate fails.
- samples: writes the ten synthetic images with fake identifiers.
- demo: registers a patient, uploads a burned-in X-ray, analyses it, shows the agent being refused, signs off as the clinician, writes the PDF and verifies the audit chain.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
def cmd_serve(a)
def cmd_ingest(a)
def cmd_eval(a)
def cmd_samples(a)
def cmd_demo(a) : Run one full end-to-end case in the terminal (no server needed).
def main(argv=None)
```

5.5 Retrieval (medrag/retrieval/)

Turns guideline text into searchable indexes and finds the evidence for a question.

`medrag/retrieval/__init__.py` · 5 lines

What it is. Exposes RetrievalPipeline and SearchHit.

Why it was created. Gives other modules a short, stable import path.

`medrag/retrieval/chunking.py` · 126 lines

What it is. Reads knowledge-base files and cuts them into overlapping passages.

Why it was created. Passages of about 500 tokens are small enough to be specific and large enough to carry context. Cutting at sentence boundaries keeps every passage readable and quotable. Overlap prevents losing a fact that spans a boundary.

What it does

- Document and Chunk dataclasses. Chunk.payload() gives only the allow-listed fields stored in the vector database.
- chunk_text(): accumulates sentences up to max_tokens, then starts the next chunk with the last sentences (up to the overlap size).
- parse_markdown(): reads the front matter and strips headings and bullets, keeping the words.
- load_directory(): parses every .md file in a folder. doc_id_for(): a stable id for uploaded documents.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class Document
class Chunk
  .payload(self)
def _ntok(s: str)
def chunk_text(text: str, max_tokens: int=500, overlap_tokens: int=60)
def chunk_document(doc: Document, max_tokens: int=500, overlap_tokens: int=60)
const _FRONT_RE
def parse_markdown(text: str, fallback_id: str) : Parse a markdown file with an optional simple ``key: value`` front matter.
def load_directory(path: Path)
def doc_id_for(text: str)
```

`medrag/retrieval/embeddings.py` · 70 lines

What it is. Converts text into vectors for semantic (dense) search.

Why it was created. BGE-M3 is a strong multilingual retrieval encoder. The offline HashingEmbedder lets the dense half of the pipeline run deterministically with no model download.

What it does

- HashingEmbedder: hashes stemmed words, word pairs and character trigrams into a 512-dimension vector with ±1 signs, then normalises it. Related wording produces similar vectors.
- BGEM3Embedder: loads BAAI/bge-m3 through sentence-transformers the first time it is used (lazy loading), and returns normalised embeddings.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class Embedder
  .embed(self, texts: list[str])
def _h(s: str, dim: int)
class HashingEmbedder
  .__init__(self, dim: int=512)
  ._one(self, text: str)
  .embed(self, texts: list[str])
class BGEM3Embedder
  .__init__(self, model_name: str='BAAI/bge-m3')
  ._load(self)
  .embed(self, texts: list[str])
```

medrag/retrieval/sparse.py · 59 lines

What it is. A BM25 keyword index: the sparse half of hybrid retrieval.

Why it was created. Dense search can miss exact terms such as drug names, "CURB-65" or "Xpert MTB/RIF". BM25 rewards exact matches, weighted by rarity.

What it does

- add() / remove(): maintain term frequencies, document lengths and document frequencies.
- search(): standard Okapi BM25 scoring (k1 = 1.5, b = 0.75), optionally only over allowed chunk ids (the metadata filter).

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class BM25Index
  .__init__(self, k1: float=1.5, b: float=0.75)
  .add(self, item_id: str, text: str)
  .remove(self, item_id: str)
  .search(self, query: str, k: int=10, allowed: set[str] | None=None)
```

medrag/retrieval/vector_store.py · 140 lines

What it is. Stores the dense vectors and their payloads, and enforces the privacy allowlist.

Why it was created. The allowlist is a structural privacy guarantee: any payload field not explicitly permitted is rejected, so a patient identifier can never reach the vector database, whatever the caller does.

What it does

- check_payload(): raises PayloadNotAllowed for any field outside the guideline and case field lists.
- InMemoryVectorStore: exact cosine search with NumPy. Supports upsert, get, search with an allowed set, and save/load as .npy plus .json.
- QdrantVectorStore: the same interface backed by Qdrant for production. It also applies the allowlist.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const GUIDELINE_FIELDS
const CASE_FIELDS
const ALLOWED_PAYLOAD_FIELDS
class PayloadNotAllowed
def check_payload(payload: dict)
class InMemoryVectorStore : Exact cosine search over a numpy matrix; persisted as .npy + .json.
  .__init__(self, dim: int)
  .upsert(self, item_id: str, vector: np.ndarray, payload: dict)
  .get(self, item_id: str)
  .search(self, vector: np.ndarray, k: int=10, allowed: set[str] | None=None)
  .save(self, path: Path)
  .load(cls, path: Path, dim: int)
class QdrantVectorStore : Same interface backed by Qdrant (used in real / containerised mode).
  .__init__(self, dim: int, url: str, collection: str='medrag_guidelines')
  ._pid(item_id: str)
  .upsert(self, item_id, vector, payload)
  .get(self, item_id)
  .search(self, vector, k=10, allowed=None)
  .save(self, path)
```

medrag/retrieval/fusion.py · 11 lines

What it is. Reciprocal rank fusion (RRF).

Why it was created. RRF merges the dense and sparse rankings without needing their scores on the same scale. It is simple, robust and standard in hybrid retrieval.

What it does

- score(item) = sum over rankings of 1 / (k + rank), with k = 60. Ties are broken by id so results are deterministic.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
def reciprocal_rank_fusion(rankings: list[list[str]], k: int=60)
```

medrag/retrieval/rerank.py · 54 lines

What it is. Re-scores the fused candidates jointly with the query.

Why it was created. A cross-encoder reads the query and passage together and is more precise than either retriever alone. Better precision means fewer agent loops.

What it does

- LexicalReranker (offline): weighted query-term coverage plus matched word pairs plus term density, imitating a cross-encoder.
- CrossEncoderReranker (real): BAAI/bge-reranker-v2-m3, loaded lazily.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class Reranker
  .score(self, query: str, passages: list[str])
class LexicalReranker
  .score(self, query: str, passages: list[str])
class CrossEncoderReranker
  .__init__(self, model_name: str='BAAI/bge-reranker-v2-m3')
  .score(self, query, passages)
```

medrag/retrieval/pipeline.py · 191 lines

What it is. Orchestrates the full search: filter, dense, sparse, fuse, re-rank.

Why it was created. Keeping both indexes in one class guarantees they are always written together and never drift apart. The strategy switch lets the evaluation compare dense, sparse, hybrid and hybrid + re-rank.

What it does

- add_documents(): replaces old chunks of the same document, embeds the new chunks, and writes them to both indexes.
- _allowed(): the metadata pre-filter. It keeps chunks whose modality matches (or is general) and whose age band matches.
- search(): takes the top 20 candidates from each retriever, fuses them with RRF, re-ranks, and returns the top K SearchHits with their dense and sparse ranks and per-stage timings in milliseconds.
- save() / load(): persist the index to var/index.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class SearchHit
  .to_dict(self)
class SearchResult
const STRATEGIES
class RetrievalPipeline
  .__init__(self, settings: Settings | None=None, embedder=None, reranker=None…)
  .add_documents(self, docs: list[Document])
  .ingest_directory(self, path: Path | None=None)
  .stats(self)
  ._allowed(self, modality: str | None, age_band: str | None)
  .search(self, query: str, k: int | None=None, modality: str | None=None, a…)
  .save(self, path: Path | None=None)
  .load(self, path: Path | None=None)
```

5.6 Imaging (medrag/imaging/)

Everything that looks at pictures.

`medrag/imaging/__init__.py` · 1 lines

What it is. Package marker.

`medrag/imaging/synthetic.py` · 127 lines

What it is. Generates deterministic cartoon medical images, optionally with fake identifiers burned in.

Why it was created. Real datasets need credentialing and cannot be shipped. Synthetic images let the whole pipeline, the de-identification step and the tests run anywhere, reproducibly (fixed seeds).

What it does

- chest_xray(): lung fields, spine, heart shadow. Variants add lower-zone blobs (consolidation), a basal meniscus (effusion), a wide heart (cardiomegaly) or a dark apex (pneumothorax).
- fundus(): retina, vessels, optic disc and cup. Variants add haemorrhages and exudates (diabetic retinopathy) or a large cup (glaucoma).
- histopathology(): pink stroma with nuclei. The carcinoma variant has dense nuclei of very different sizes.
- _burn(): writes "NAME: JOHN DOE MRN ..." and an accession number into the pixels. to_png_bytes() can also add metadata text chunks.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const FAKE_BURNED_TEXT
def _burn(img: Image.Image, text: str, fill, box_fill=None)
def chest_xray(finding: str='normal', seed: int=0, burn_text: bool=False, size: i…) : finding in {normal,
  consolidation, effusion, cardiomegaly, pneumothorax}.
def fundus(finding: str='normal', seed: int=0, burn_text: bool=False, size: i…) : finding in {normal,
  diabetic_retinopathy, glaucoma}.
def histopathology(finding: str='benign', seed: int=0, burn_text: bool=False, size: i…) : finding in {benign,
  carcinoma}.
const GENERATORS
def to_png_bytes(img: Image.Image, **info: str)
```

medrag/imaging/analysis.py · 161 lines

What it is. The deterministic image analyser used as the offline vision model.

Why it was created. It produces real, reproducible findings from pixel measurements, so the agent's image path can be tested without a GPU. It is clearly labelled a stand-in; real mode replaces it with Qwen2.5-VL.

What it does

- Finding dataclass: text, concept, location, confidence, source = image.
- Chest X-ray: compares left and right lung zones (patient right is image left) for opacity, checks the costophrenic bases for effusion and the apices for pneumothorax, and estimates the cardiothoracic ratio (above 0.5 means cardiomegaly).
- Fundus: counts dark-red blobs (haemorrhages) and yellow foci (exudates) inside the retina, and computes the cup-to-disc ratio (0.6 or more is suspicious for glaucoma).
- Histopathology: measures nuclear area fraction and size variation (pleomorphism) to separate carcinoma-like from benign-like tissue.
- _components(): a small connected-component counter used for blobs and nuclei.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class Finding
  .to_dict(self)
def _components(mask: np.ndarray, min_size: int=3) : Sizes of 4-connected components in a boolean mask (pure
  Python BFS).
def _zone(a: np.ndarray, x0, x1, y0, y1)
def analyze_chest_xray(img: Image.Image)
def analyze_fundus(img: Image.Image)
def analyze_histopathology(img: Image.Image)
const ANALYZERS
def analyze(img: Image.Image, modality: str)
```

medrag/imaging/vlm.py · 52 lines

What it is. The vision-language model interface.

Why it was created. The agent depends only on describe(image, modality, question) returning findings, so the offline analyser and Qwen2.5-VL are interchangeable.

What it does

- OfflineVLM: wraps analysis.analyze().
- QwenVLM: sends the de-identified image and a strict prompt ("list only visible findings, do not diagnose beyond them, return JSON") to the served model and parses the reply.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const MODALITY_NAMES
class VisionModel
  .describe(self, image_png: bytes, modality: str, question: str='')
class OfflineVLM
  .describe(self, image_png: bytes, modality: str, question: str='')
const VLM_PROMPT
class QwenVLM
  .__init__(self, client: ChatClient | None=None)
  .describe(self, image_png: bytes, modality: str, question: str='')
```

`medrag/imaging/encoder.py` · 71 lines

What it is. Image embeddings and the similar-case index.

Why it was created. Showing similar prior cases, signed off by clinicians, gives useful visual context. The index stores only allow-listed, de-identified fields.

What it does

- ThumbnailEncoder (offline): a 32×32 grey thumbnail plus colour histograms, normalised.
- BiomedCLIPEncoder (real): Microsoft BiomedCLIP through open_clip, loaded lazily.
- CaseIndex: add() stores study id, modality, age band and finding summary. similar() returns the closest cases of the same modality, excluding the study itself.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class ThumbnailEncoder
  .encode(self, image_png: bytes)
class BiomedCLIPEncoder
  .__init__(self, model_name: str)
  .encode(self, image_png: bytes)
class CaseIndex : Similar prior-case search over de-identified studies.
  .__init__(self, encoder)
  .add(self, study_id: str, image_png: bytes, modality: str, age_band: st…)
  .similar(self, image_png: bytes, modality: str, k: int=3, exclude: str | No…)
```

5.7 De-identification (medrag/deid/)

Removes patient identifiers before anything is stored.

`medrag/deid/__init__.py` · 7 lines

What it is. Exposes the public de-identification functions.

Why it was created. One import for the service layer.

`medrag/deid/hashing.py` · 28 lines

What it is. Patient pseudonyms and age bands.

Why it was created. Records for one patient must still be linkable, but the real ID must never be stored. A keyed HMAC is stable, irreversible without the secret salt, and resistant to guessing by dictionary. Exact ages, and especially ages over 89, can identify people, so they are banded.

What it does

- pseudonymize(): HMAC-SHA256(salt, normalised ID) gives "pt_" plus 32 hex characters. Empty IDs are rejected.
- age_band(): 0-17, 18-39, 40-64, 65+ or unknown. Impossible ages are rejected.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
def pseudonymize(patient_id: str, salt: str) : Keyed HMAC-SHA256: stable for linkage, irreversible without the
  secret salt.
def age_band(age: int | None)
```

medrag/deid/text.py · 47 lines

What it is. Rule-based scrubbing of personal health information (PHI) in free text.

Why it was created. Clinical notes and questions often contain names, record numbers and phone numbers. Regular expressions cover the HIPAA Safe-Harbour categories that appear in text, plus the Pakistani CNIC format.

What it does

- PATTERNS: email, URL, IP address, CNIC, SSN, MRN or accession number, phone, dates (numeric and written), names after a label or title (Patient:, Mr, Dr ...), street addresses, and ages of 90 or over.
- scrub_text(): replaces each match with [CATEGORY] and returns counts per category (stored in the study's de-identification summary).
- contains_phi(): a quick yes/no check.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const _NAME
def scrub_text(text: str) : Return (scrubbed_text, counts_per_category).
def contains_phi(text: str)
```

medrag/deid/image.py · 136 lines

What it is. Image de-identification: metadata removal and burned-in text masking.

Why it was created. Identifiers hide in two places: file metadata (EXIF, PNG text, DICOM headers) and text drawn into the pixels. Re-encoding from raw pixels removes the first; the stroke detector removes the second without damaging the diagnostic image (tested).

What it does

- _load(): decodes PNG/JPEG (or DICOM with pydicom) and lists the metadata keys that will be dropped.
- detect_text_tiles(): compares each pixel with its local average to find thin, very bright or very dark strokes. An 8×8 tile is a seed when at least two of its rows contain three or more stroke edges and a neighbouring tile is also a seed (text runs horizontally). Seeds grow along text lines through weaker candidate tiles, bridging gaps between words, and the result is dilated by one tile.
- _ocr_boxes(): if Tesseract is installed (real mode), its word boxes are masked too.
- deidentify_image(): blacks out the detected regions, builds a brand-new image from the pixels and saves it as PNG. It returns the PNG plus a summary (regions masked, metadata removed, OCR used).

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const TILE
class ImageDeidResult
def _load(data: bytes)
def _stroke_rows(mask: np.ndarray) : For each tile: number of rows that contain >=4 foreground/background
  transitions.
def detect_text_tiles(img: Image.Image)
def _ocr_boxes(img: Image.Image)
def deidentify_image(data: bytes, use_ocr: bool=True)
```

5.8 Agent (medrag/agent/)

The reasoning workflow that drafts and checks the report.

medrag/agent/__init__.py · 6 lines

What it is. Exposes MedRAGAgent, AgentConfig and AgentState.

Why it was created. Short import path.

medrag/agent/state.py · 79 lines

What it is. The data that flows through the agent.

Why it was created. An explicit state object makes every step inspectable. The trace becomes the "Agent trace" panel in the UI and is stored with the report for audit.

What it does

- Claim: text, section (findings, impression, differential, recommendations), citation numbers, concept, source (image or evidence), and supported flag and score.
- Draft: list of claims, summary, generator name. Verification: supported and unsupported counts, unsupported texts, hallucination rate, passed, missing concepts.
- AgentState: question, modality, age band, image, needs_image, queries tried, evidence, findings, similar cases, draft, verification, iterations, status and trace. result() turns it into JSON for storage.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class Claim
class Draft
  .section(self, name: str)
  .to_dict(self)
class Verification
class AgentState
  .log(self, node: str, **info)
  .result(self)
```

medrag/agent/generator.py · 202 lines

What it is. Writes the draft report and answers chat questions.

Why it was created. Drafting is where hallucination happens, so every non-image claim must carry citations. The offline generator is extractive: it quotes evidence sentences, which keeps it honest by design. The real-mode generator is prompted to use only the numbered evidence.

What it does

- _best(): picks the best unused evidence sentence for a query. It can require cue words ("differential", "recommend", "should"...), require that a finding concept is present and not negated ("no nuclear pleomorphism" does not count), and prefers sentences that name the exact finding.
- ExtractiveGenerator.draft(): image findings become finding claims. Each abnormal finding gets a cited impression sentence. If no evidence supports a finding yet, an uncited claim is written on purpose, which makes the verifier trigger another retrieval. A differential (only for abnormal findings) and up to two recommendations follow.
- ExtractiveGenerator.answer(): up to three cited sentences for the chat assistant.
- LLMGenerator (real): JSON-returning prompt. Citations that point outside the evidence list are discarded.
- claim_support(): the best overlap between a claim and its cited passages.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const RECOMMEND_CUES
const DIFFERENTIAL_CUES
const _NEGATIONS
def _negated(sentence: str, concept: str) : True if the concept only occurs in a negated phrase, e.g. "no nuclear
  pleomorphism".
class Generator
  .draft(self, question: str, evidence: list[dict], findings: list[Finding]…)
  .answer(self, question: str, evidence: list[dict])
def _sentences(evidence: list[dict])
def _best(sentences: list[tuple[int, str]], query: str, used: set[str], cues…) : Pick the unused evidence
  sentence that best matches ``query``.
class ExtractiveGenerator
  .draft(self, question, evidence, findings, similar)
  .answer(self, question, evidence)
const DRAFT_PROMPT
class LLMGenerator
  .__init__(self, client: ChatClient | None=None)
  ._ev(evidence)
  .draft(self, question, evidence, findings, similar)
  .answer(self, question, evidence)
def claim_support(claim: Claim, evidence: list[dict]) : Best lexical support score of a claim against its cited
  passages.
```

medrag/agent/verifier.py · 49 lines

What it is. The critic that checks every claim.

Why it was created. Self-verification is the core of RQ4. A claim counts as supported only when it cites evidence AND shares at least 50% of its key terms with a cited passage.

What it does

- check(): image claims are supported by the image. Every other claim is scored and marked. It computes the hallucination rate and collects the concepts of unsupported claims.
- reformulate(): builds the next query from the missing concepts plus "diagnosis management guideline", avoiding repeating a previous query.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class Verifier
  .__init__(self, threshold: float=0.5)
  .check(self, draft: Draft, evidence: list[dict])
  .reformulate(question: str, verification: Verification, previous: list[str]) : Build a follow-up query
    focused on what the draft could not support.
```

medrag/agent/graph.py · 177 lines

What it is. The agent itself: an explicit state machine with five nodes.

Why it was created. A plain, dependency-free loop makes the control flow easy to read, test and audit. The same graph can be compiled with LangGraph (build_langgraph). The agent's best result is a verified draft; signing off lies outside it.

What it does

- AgentConfig and its presets: full, no_retrieval, single_pass, text_only (the ablation variants), max_iterations and top_k.
- route(): infers the modality from words in the question if needed, and decides whether the image is used.
- retrieve(): hybrid search with modality and age filters, adding only new evidence.

- inspect_image(): VLM findings plus similar cases. Before the first draft, a second retrieval is run with the finding concepts added to the question.
- draft(): increments the iteration count and calls the generator.
- verify(): if passed, status is verified. Without self-verification, status is unverified. After the maximum number of iterations, status is needs_review and the draft is flagged. Otherwise it reformulates the query and loops back to retrieve.
- run(): executes nodes until END, with a hard step budget against runaway loops. ask(): retrieval plus a cited answer for the chat.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const IMAGE_TERMS
const MODALITY_TERMS
class AgentConfig
  .variant(cls, name: str, **kw)
class MedRAGAgent
  .__init__(self, pipeline: RetrievalPipeline, settings: Settings | None=None,…)
  .route(self, st: AgentState, cfg: AgentConfig)
  .retrieve(self, st: AgentState, cfg: AgentConfig)
  .inspect_image(self, st: AgentState, cfg: AgentConfig)
  .draft(self, st: AgentState, cfg: AgentConfig)
  .verify(self, st: AgentState, cfg: AgentConfig)
  .run(self, question: str, *, modality: str | None=None, age_band: str |…)
  .ask(self, question: str, modality: str | None=None, k: int=5)
def build_langgraph(agent: MedRAGAgent, cfg: AgentConfig) : Same control flow compiled with LangGraph (``pip
  install langgraph``).
```

5.9 Reports (medrag/reports/)

The rules for finalising a report, and the document itself.

medrag/reports/__init__.py · 5 lines

What it is. Exposes Actor, ReportStatus, SignOffError and transition.

Why it was created. Short import path.

medrag/reports/lifecycle.py · 72 lines

What it is. The report state machine that enforces human sign-off.

Why it was created. This is the structural oversight guarantee from the synopsis. The check sits below the API, so every caller goes through it.

What it does

- States: draft, verified, needs_review, signed_off, rejected. The last two are terminal and can never change.
- Actor: id, kind (human, agent or system) and role. is_clinician is true only for a human with the clinician role.
- transition(): checks that the move is allowed. Only a human clinician may move to signed_off or rejected, and a reason is required to reject. Humans may not set verification states; only the agent records those.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class ReportStatus
const TERMINAL
const HUMAN_ONLY
class SignOffError
class Actor
  .is_clinician(self)
def transition(current: ReportStatus | str, target: ReportStatus | str, actor: Ac…)
```

`medrag/reports/document.py` · 114 lines

What it is. Renders the report as a PDF with ReportLab.

Why it was created. Clinicians need a portable document. The watermark makes the status of an unsigned draft impossible to miss.

What it does

- Header table (report and study ids, status, modality, pseudonym, age band, agent status, iterations), the clinical question and the image.
- Sections with bullet claims, citation numbers, (image) and (unsupported) markers, then the cited evidence list and the verification summary.
- Sign-off block: signer and time, the rejection reason, or "NOT SIGNED".
- Every page carries a footer. Unsigned reports also get the diagonal "DRAFT - NOT FOR CLINICAL USE" watermark.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const WATERMARK
const SECTIONS
def _watermark(canvas, doc)
def _footer(canvas, doc)
def render_report_pdf(report: dict, study: dict, image_png: bytes | None=None) : ``report``: row dict with
  ``status``, ``content`` (agent result), sign-off fields.
```

5.10 Storage (medrag/storage/)

Where data lives.

`medrag/storage/__init__.py` · 4 lines

What it is. Exposes Database and ObjectStore.

Why it was created. Short import path.

`medrag/storage/db.py` · 217 lines

What it is. The relational database (SQLite) and the tamper-evident audit log.

Why it was created. SQLite needs no server, which suits a research prototype. A hospital would swap in PostgreSQL behind the same methods. The audit log must be trustworthy, so it is protected twice.

What it does

- Tables: users (role limited to clinician, admin or viewer), patients (pseudonym, age band, sex), studies (scrubbed note, image key, de-identification summary), reports (status, JSON content, signer, comment), audit_log.
- Triggers abort any UPDATE or DELETE on audit_log.
- audit(): each entry stores the SHA-256 of its fields plus the previous entry's hash (a hash chain). verify_audit_chain() recomputes the chain and reports where it breaks.

- Passwords use PBKDF2-SHA256 with 200,000 iterations and a random salt, compared in constant time.
- Methods for creating and reading users, patients, studies and reports. set_report_status() is a low-level write that must only be called after lifecycle.transition().

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const SCHEMA
def now()
def hash_password(password: str, salt: str | None=None)
def check_password(password: str, stored: str)
class Database
  .__init__(self, path: Path | str)
  ._q(self, sql: str, args=())
  ._x(self, sql: str, args=())
  .create_user(self, username: str, password: str, role: str, full_name: str)
  .get_user(self, username: str)
  .authenticate(self, username: str, password: str)
  .count_users(self)
  .upsert_patient(self, patient_ref: str, age_band: str, sex: str)
  .list_patients(self)
  .get_patient(self, ref: str)
  .create_study(self, **f)
  .get_study(self, sid: str)
  .list_studies(self, patient_ref: str | None=None)
  .create_report(self, study_id: str | None, status: str, content: dict, created_by…)
  .get_report(self, rid: str)
  .list_reports(self, study_id: str | None=None)
  .set_report_status(self, rid: str, status: str, signed_by: str | None=None, comment: …) : Low-level write.
    Callers MUST go through ``reports.lifecycle.transition`` first.
  ._row_hash(ts, actor, kind, action, entity, details, prev)
  .audit(self, actor: str, actor_kind: str, action: str, entity: str, detai…)
  .audit_entries(self, limit: int=200, entity: str | None=None)
  .verify_audit_chain(self)
```

medrag/storage/objects.py · 37 lines

What it is. A content-addressed file store for de-identified images.

Why it was created. Images are named by their SHA-256 hash, so identical uploads are stored once and file names reveal nothing. Strict key validation blocks path-traversal attacks. The interface can be backed by MinIO or S3 in production.

What it does

- put(), get() and exists(), with key format checking.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const _KEY_RE
class ObjectStore
  .__init__(self, root: Path)
  .put(self, data: bytes, prefix: str, ext: str)
  .get(self, key: str)
  .exists(self, key: str)
  ._path(self, key: str)
```

5.11 Web API (medrag/api/)

The secured HTTP interface.

`medrag/api/__init__.py` · 3 lines

What it is. Exposes create_app.

Why it was created. Short import path.

`medrag/api/auth.py` · 52 lines

What it is. Login tokens and role permissions.

Why it was created. Stateless signed tokens need no session table. A permission table makes the access rules explicit and reviewable in one place.

What it does

- issue_token() / verify_token(): HMAC-SHA256-signed payload (user, role, expiry), with signature and expiry checks.
- PERMISSIONS: read (all roles), ingest (clinician, admin), analyze and sign_off (clinician only), kb_write and audit (admin only).

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
class TokenError
def _b64(b: bytes)
def _unb64(s: str)
def issue_token(username: str, role: str, secret: str, ttl: int)
def verify_token(token: str, secret: str)
const PERMISSIONS
```

`medrag/api/app.py` · 241 lines

What it is. The FastAPI application: request models, endpoints and the static web app.

Why it was created. FastAPI validates every request automatically and generates interactive API documentation at /docs. The endpoints are thin and delegate to the service, so all rules live in one place.

What it does

- Pydantic request models validate input: patient age 0-130, question length, allowed variants and strategies, and so on.
- current_user() reads the Bearer token. require(permission) checks the role and audits denials.
- Endpoints: health, login, me, patients, studies (upload, list, get, image, analyze), reports (list, get, sign off, reject, PDF), search, ask, knowledge base (stats, add), synthetic samples, audit (list, verify).
- Errors are translated: not found gives 404, sign-off refusal gives 403, invalid input gives 400, and uploads over 20 MB give 413.
- Serves the web workspace at / and /static.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const WEB_DIR
const MAX_UPLOAD
class LoginIn
class PatientIn
class AnalyzeIn
class AskIn
class SearchIn
class DecisionIn
class KBDocIn
def create_app(settings: Settings | None=None, service: MedRAGService | None=None)
  GET /api/health : handler: health()
  POST /api/auth/login : handler: login()
  GET /api/me : handler: me()
  POST /api/patients : handler: create_patient()
  GET /api/patients : handler: list_patients()
  GET /api/studies : handler: list_studies()
  GET /api/studies/{sid} : handler: get_study()
  GET /api/studies/{sid}/image : handler: study_image()
  POST /api/studies/{sid}/analyze : handler: analyze()
  GET /api/reports : handler: list_reports()
  GET /api/reports/{rid} : handler: get_report()
  POST /api/reports/{rid}/signoff : handler: sign_off()
  POST /api/reports/{rid}/reject : handler: reject()
  GET /api/reports/{rid}/pdf : handler: report_pdf()
  POST /api/search : handler: search()
  POST /api/ask : handler: ask()
  GET /api/kb : handler: kb_stats()
  POST /api/kb/documents : handler: kb_add()
  GET /api/samples : handler: samples()
  GET /api/samples/{modality}/{finding} : handler: sample_image()
  GET /api/audit : handler: audit()
  GET /api/audit/verify : handler: audit_verify()
  GET / : handler: index()
```

5.12 Evaluation (medrag/eval/)

Measures how well the system works.

medrag/eval/__init__.py · 1 lines

What it is. Package marker.

medrag/eval/metrics.py · 66 lines

What it is. Metric definitions exactly as specified in the synopsis.

Why it was created. Small, separately tested functions make the reported numbers trustworthy.

What it does

- recall_at_k(), reciprocal_rank(), mean(), p95(), hallucination_rate().
- tfidf_cosine(): offline report-similarity stand-in. SemanticSimilarity: a biomedical sentence encoder for real mode.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
def recall_at_k(retrieved: list[str], relevant: set[str], k: int) : |relevant ∩ top-k| / |relevant|.
def reciprocal_rank(retrieved: list[str], relevant: set[str])
def mean(xs)
def p95(xs)
def hallucination_rate(supported_flags: list[bool]) : Unsupported claims / total claims (lower is better).
def tfidf_cosine(a: str, b: str, corpus: list[str] | None=None) : Offline stand-in for semantic report similarity.
class SemanticSimilarity
  .__init__(self, model_name: str)
```

`medrag/eval/runner.py` · 124 lines

What it is. Runs the full evaluation and writes the results.

Why it was created. One reproducible command produces every table needed for the thesis, and the release gates turn quality into a pass or fail check for CI.

What it does

- evaluate_retrieval(): Recall@1/3/5, MRR, and mean and P95 latency for each of the four strategies.
- evaluate_clinical(): for each variant and gold case, generates the image, runs the agent, then measures the ungrounded-claim rate, similarity, finding accuracy, verified rate, iterations and latency.
- THRESHOLDS and the gates. to_markdown() and run() write eval_results/results.json and results.md.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
const GOLD_DIR
const VARIANTS
const THRESHOLDS
def evaluate_retrieval(pipeline: RetrievalPipeline, gold: list[dict], ks=(1, 3, 5))
def _report_text(state)
def evaluate_clinical(agent: MedRAGAgent, cases: list[dict], similarity=None)
def to_markdown(results: dict)
def run(settings: Settings | None=None, out_dir: Path | None=None)
```

5.13 Web workspace (medrag/web/)

The clinician-facing user interface. It is plain HTML, CSS and JavaScript, with no build step.

`medrag/web/index.html` · 158 lines

What it is. The page structure: header and tabs, disclaimer, login, the three-column workspace, the knowledge-base view and the audit view.

Why it was created. A single static page served by the API is simple to deploy offline, with no Node.js build required on hospital servers.

What it does

- Workspace columns: patients and new-study forms; study image, de-identification summary, question with variant selector and assistant chat; and the draft report with sign-off, reject and PDF buttons.

`medrag/web/style.css` · 63 lines

What it is. Visual styling.

Why it was created. Status colours (verified green, needs review amber, rejected red), visibly flagged unsupported claims and citation chips make safety-relevant information obvious at a glance.

What it does

- Responsive grid that stacks to one column on narrow screens.

`medrag/web/app.js` · 268 lines

What it is. The client logic.

Why it was created. Talks only to the documented API with the user's token, and hides buttons the role cannot use. The server still enforces every permission.

What it does

- Login and logout, with the token stored in the browser. Permission-based visibility of forms and buttons.
- Register patients, upload a file or a synthetic sample, show the de-identified image and its summary.
- Run an analysis and render the report: sections, citation chips with the evidence shown on hover, red unsupported claims, the verification line, evidence, similar cases, agent trace and version history.
- Sign off (with a confirmation), reject, open the PDF. Assistant chat with citations. Knowledge-base search showing dense and sparse ranks and timings. Admin document upload. Audit log table and hash-chain verification.

5.14 Tests (tests/)

117 automated tests that prove the system behaves as designed. Run with make test.

`tests/conftest.py` · 40 lines

What it is. Shared test fixtures.

Why it was created. Every test gets an isolated temporary data folder and fixed secrets, so tests never interfere with each other or with real data.

What it does

- settings, pipeline (shared and read-only), service, client (FastAPI TestClient), and the login() helper.

Code index: every class, method, function, constant and endpoint in this file, extracted automatically from the source (docstrings shown in grey)

```
def settings(tmp_path)
def pipeline()
def service(settings)
def client(settings)
def login(client, user='clinician', pw=None)
```

`tests/test_retrieval.py` · 121 lines

What it is. Tests text utilities, chunking, embedders, BM25, RRF, the re-ranker, the vector-store allowlist, top-1 accuracy, the modality filter, all strategies and persistence.

Why it was created. Proves objective O1's building blocks.

Tests in this file (16): test_stem_and_overlap, test_split_sentences, test_chunking_respects_sentences_and_size, test_chunking_overlap_repeats_trailing_sentence, test_parse_markdown_front_matter, test_hashing_embedder_deterministic_and_normalised, test_bm25_ranks_exact_terms_first, test_rrf_fusion, test_reranker_prefers_full_query_coverage, test_vector_store_rejects_non_allowlisted_payload, test_vector_store_dimension_check, test_hybrid_search_top1, test_modality_filter, test_all_strategies_return_ranked_hits, test_unknown_strategy, test_pipeline_persistence_and_reingest

`tests/test_deid.py` · 84 lines

What it is. Tests every PHI category, pseudonym properties, age bands, metadata removal, burned-in text removal with no false positives, preserved diagnostic content and bad input.

Why it was created. Proves the privacy half of objective O3.

Tests in this file (8): test_scrub_text_categories, test_scrub_keeps_clinical_content, test_pseudonym_stable_irreversible_and_salted, test_age_band, test_image_metadata_removed, test_burned_in_text_removed_without_false_positives, test_deid_preserves_diagnostic_content, test_invalid_image_raises

tests/test_imaging.py · 39 lines

What it is. Tests that all ten synthetic findings are detected across seeds, unknown modalities, and that similar-case search returns the same pathology.

Why it was created. Proves the image path used by objective O2.

Tests in this file (3): test_offline_vlm_findings, test_unknown_modality, test_similar_case_index_finds_same_pathology

tests/test_agent.py · 141 lines

What it is. Tests node order, citations, image and no-image routes, all variants, recovery of the self-correction loop, the bounded loop (1, 2 and 3 iterations), the verifier, reformulation, negation and the chat answer.

Why it was created. Proves objective O4 and the agent control flow, using deliberately hallucinating generators.

Tests in this file (14): test_full_flow_order_and_verified, test_every_non_image_claim_cites_valid_evidence, test_no_image_skips_inspection, test_text_only_variant_ignores_image, test_no_retrieval_baseline_is_ungrounded, test_self_verification_loop_recovers, test_single_pass_does_not_loop, test_loop_is_bounded_and_flags_for_review, test_verifier_flags_uncited_and_mismatched_claims, test_reformulate_targets_missing_concepts, test_negation_detection, test_route_infers_modality, test_variant_presets, test_ask_returns_cited_answer

tests/test_reports.py · 75 lines

What it is. Tests every forbidden sign-off path (agent, system, admin, viewer), terminal states, the rejection reason, forged verification, and PDF watermarks.

Why it was created. Proves the oversight half of objective O3.

Tests in this file (9): test_agent_can_record_verification, test_non_human_can_never_sign_off, test_non_clinician_humans_cannot_sign_off, test_clinician_signs_off_and_terminal_is_final, test_reject_requires_reason, test_humans_cannot_forge_verification, test_illegal_transition, test_unsigned_pdf_has_watermark, test_signed_pdf_has_no_watermark

tests/test_storage.py · 51 lines

What it is. Tests password hashing, the append-only triggers, tamper detection by the hash chain, object-key validation, and users and reports.

Why it was created. Proves the audit guarantees.

Tests in this file (5): test_password_hashing, test_audit_log_is_append_only, test_audit_chain_detects_tampering, test_object_store_roundtrip_and_key_validation, test_users_and_reports

tests/test_service.py · 124 lines

What it is. End-to-end service tests, including a scan of every stored byte for the raw identifiers, agent refusal, clinician sign-off, admin refusal, knowledge-base upload, similar cases and index rebuild.

Why it was created. Proves that the guarantees hold when everything runs together.

Tests in this file (11): test_no_identifiable_data_is_stored, test_patient_identifier_stored_only_as_hash, test_analyze_creates_verified_draft_never_signed, test_agent_cannot_sign_off_and_attempt_is_audited, test_clinician_sign_off_flow_and_audit_chain, test_admin_cannot_sign_off, test_unknown_patient_and_modality, test_audit_table_append_only_via_service, test_kb_document_ingestion, test_similar_cases_after_signoff, test_index_rebuilds_when_kb_changes_and_keeps_uploads

tests/test_api.py · 99 lines

What it is. HTTP tests: health, authentication, the full clinical workflow, viewer and admin restrictions, the rejection reason, search, ask and knowledge base, validation and samples.

Why it was created. Proves that the role-based access control works over HTTP.

Tests in this file (9): test_health_and_index, test_login_failure_and_missing_token, test_full_clinical_workflow, test_viewer_is_read_only, test_admin_cannot_sign_off_but_can_audit, test_reject_needs_reason, test_search_ask_and_kb, test_validation_errors, test_samples_endpoint

tests/test_eval.py · 34 lines

What it is. Tests the metric formulas, that gold data is disjoint from the knowledge base, retrieval evaluation, and that the full run beats the ablations and passes the gates.

Why it was created. Proves the evaluation harness is correct.

Tests in this file (4): test_metric_definitions, test_gold_sets_are_disjoint_from_kb, test_retrieval_eval, test_full_eval_run_and_ablation_ordering

6. Database tables

Table	Columns	Notes
users	username, full_name, role, password_hash, created_at	role limited to clinician / admin / viewer; PBKDF2 hashes
patients	patient_ref, age_band, sex, created_at	patient_ref is the HMAC pseudonym; no name, no raw ID, no birth date
studies	id, patient_ref, modality, age_band, clinical_note, image_key, width, height, deid_summary, created_by, created_at	clinical_note is PHI-scrubbed; image_key points to the de-identified PNG
reports	id, study_id, status, content (JSON), created_by, created_at, signed_by, signed_at, comment	content holds the full agent result incl. evidence and trace
audit_log	seq, ts, actor, actor_kind, action, entity, details, prev_hash, hash	UPDATE/DELETE blocked by triggers; SHA-256 hash chain

Files written at run time (inside var/)

- **medrag.sqlite3**: the database above.
- **objects/studies/<sha256>.png**: de-identified images.
- **index/**: dense vectors (vectors.npy, payloads.json), chunks.json and kb_fingerprint.
- **kb_uploads/**: guideline documents added by an admin, saved as markdown.

7. API endpoints

Method & path	Who may call it	Purpose
GET /api/health	anyone	status, version, mode, index size, model names
POST /api/auth/login	anyone	returns a signed token
GET /api/me	logged in	user, role, permissions
POST /api/patients	clinician, admin	register (pseudonymise) a patient
GET /api/patients	all roles	list patients
POST /api/studies	clinician, admin	upload and de-identify an image (multipart)
GET /api/studies[/{id}][/image]	all roles	list or get studies, de-identified image
POST /api/studies/{id}/analyze	clinician	run the agent (variant: full, single_pass, text_only, no_retrieval)
GET /api/reports[/{id}]	all roles	list or get reports
POST /api/reports/{id}/signoff	clinician	sign off (human only)
POST /api/reports/{id}/reject	clinician	reject with a reason
GET /api/reports/{id}/pdf	all roles	PDF (watermarked if unsigned)
POST /api/search	all roles	hybrid search with strategy and modality
POST /api/ask	all roles	cited answer for the assistant chat
GET /api/kb · POST /api/kb/documents	all · admin	index statistics · add a guideline
GET /api/samples[/{modality}/{finding}]	all roles	synthetic demo images
GET /api/audit · /api/audit/verify	admin	audit log · hash-chain check

8. Configuration

All settings come from environment variables (copy .env.example to .env). Defaults suit offline mode.

Environment variable	Default
MEDRAG_MODE	offline
MEDRAG_SECRET_KEY	dev-secret-change-me
MEDRAG_PATIENT_SALT	dev-salt-change-me
MEDRAG_TOKEN_TTL	28800
MEDRAG_CHUNK_TOKENS	500
MEDRAG_CHUNK_OVERLAP	60
MEDRAG_TOP_K	5
MEDRAG_CANDIDATE_K	20
MEDRAG_RRF_K	60
MEDRAG_VECTOR_BACKEND	memory
MEDRAG_QDRANT_URL	http://localhost:6333
MEDRAG_MAX_ITERATIONS	3
MEDRAG_SUPPORT_THRESHOLD	0.5
MEDRAG_EMBED_MODEL	BAAI/bge-m3
MEDRAG_RERANK_MODEL	BAAI/bge-reranker-v2-m3
MEDRAG_IMAGE_ENCODER	hf-hub:microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224
MEDRAG_LLM_URL	http://localhost:8001/v1
MEDRAG_LLM_MODEL	Qwen/Qwen2.5-VL-7B-Instruct
MEDRAG_LLM_API_KEY	EMPTY
MEDRAG_LLM_TIMEOUT	120
MEDRAG_SEED_DEMO	1
MEDRAG_SIM_MODEL	pritamdeka/S-PubMedBert-MS-MARCO (evaluation, real mode)

9. How to run

Command	Effect
make install	create .venv and install the project with test tools
make run	start the server; open http://localhost:8000 (clinician / clinician123)
make test	run all tests
make lint	check code style
make eval	evaluation with release gates → eval_results/results.md
make demo	a complete case in the terminal, writes a signed PDF to var/
make samples	regenerate data/samples/
make docker	run with Docker Compose (offline)
MEDRAG_EXTRAS=real docker compose --profile gpu up --build	real mode with vLLM, Qwen2.5-VL and Qdrant on a GPU

Demo accounts: clinician / clinician123 (full clinical rights), admin / admin123 (guidelines and audit, cannot sign off), viewer / viewer123 (read-only). Change MEDRAG_SECRET_KEY and MEDRAG_PATIENT_SALT and set MEDRAG_SEED_DEMO=0 for any non-demo use.

10. How the work was verified

- **All 117 automated tests pass** and the code passes the ruff linter.
- The full clinical workflow was driven in a real browser (Chromium via Playwright): login, register, upload with burned-in identifiers, de-identification, analysis, assistant question, sign-off. There were no browser errors.
- The generated PDF was rendered and inspected, both watermarked (draft) and signed.
- De-identification was measured on 60 synthetic images: all burned-in text pixels removed, zero false positives on clean images, and findings unchanged after masking.
- The offline evaluation passes both release gates.

Offline retrieval results (harness check)

Strategy	Recall@1	Recall@3	Recall@5	MRR	Mean ms
dense	0.904	1.000	1.000	0.974	0.14
sparse	0.942	1.000	1.000	1.000	0.02
hybrid	0.904	1.000	1.000	0.981	0.12
hybrid+rerank	0.942	1.000	1.000	1.000	1.63

Offline clinical-drafting results (harness check)

Variant	Ungrounded rate	Similarity	Finding acc.	Verified	Mean ms
full	0.000	0.539	1.000	1.000	40.9
single_pass	0.000	0.539	1.000	1.000	41.9
text_only	0.000	0.211	0.643	1.000	34.5
no_retrieval	1.000	0.299	0.857	0.000	36.9

These numbers come from 26 queries and 14 synthetic cases with deterministic stand-in models. They show that the measurement code works and that the ablations behave as expected (no retrieval: all claims unsupported; text-only: lower finding accuracy). They are **not** clinical results. Offline, the full system and single-pass score the same because the extractive drafter quotes evidence word for word; the self-correction loop is proven by tests with deliberately hallucinating generators and must be measured in GPU mode.

11. Limitations and next steps

- Offline image analysis is a heuristic tuned to synthetic images; real reasoning needs GPU mode (Qwen2.5-VL).
- GPU-mode code (vLLM, BGE-M3, re-ranker, BiomedCLIP, Qdrant, Tesseract) is written but was not run, because no GPU was available; its prompts need tuning.
- The Docker files were not built in the development environment, because no Docker daemon was available.
- The knowledge base is a set of educational summaries; replace it with licensed NICE / WHO / PMC text.
- The verifier is lexical (term overlap); a natural-language-inference or LLM-judge verifier would be stronger for paraphrased text.
- DICOM is supported via optional pydicom, but full DICOM PS3.15 de-identification profiles are future work.
- **Next:** start MIMIC-CXR credentialing now, bring up GPU mode, build real gold sets, run all ablations, and get clinicians to review a sample of the drafts.